

Stacks

Example

Operation	Return Value	Stack Contents
S.push(5)	–	[5]
S.push(3)	–	[5, 3]
len(S)	2	[5, 3]
S.pop()	3	[5]
S.is_empty()	False	[5]
S.pop()	5	[]
S.is_empty()	True	[]
S.pop()	“error”	[]
S.push(7)	–	[7]
S.push(9)	–	[7, 9]
S.top()	9	[7, 9]
S.push(4)	–	[7, 9, 4]
len(S)	3	[7, 9, 4]
S.pop()	4	[7, 9]
S.push(6)	–	[7, 9, 6]
S.push(8)	–	[7, 9, 6, 8]
S.pop()	8	[7, 9, 6]

Applications of Stacks

- Page-visited history in a Web browser
- Undo/Redo sequence in a text editor
- Expression Evaluations
- Function Calls
- Backtracking Algorithms
- Auxiliary data structure for algorithms
- And many more ...

3

Array-based Stack

- A simple way of implementing the Stack ADT uses an array
- We add elements from left to right
- A variable keeps track of the index of the top element



4

Array-based Stack (cont.)

- The array storing the stack elements may become full
- A push operation will then need to grow the array and copy all the elements over.



5

Applying adapter design pattern

<i>Stack Method</i>	<i>Realization with Python list</i>
<code>S.push(e)</code>	<code>L.append(e)</code>
<code>S.pop()</code>	<code>L.pop()</code>
<code>S.top()</code>	<code>L[-1]</code>
<code>S.is_empty()</code>	<code>len(L) == 0</code>
<code>len(S)</code>	<code>len(L)</code>

Realization of a stack S as an adaptation of a Python list L

Performance of our array-based stack implementation

Operation	Running Time
S.push(e)	$O(1)^*$
S.pop()	$O(1)^*$
S.top()	$O(1)$
S.is_empty()	$O(1)$
len(S)	$O(1)$

*amortized

Array-based Stack in Python

```
1 class ArrayStack:
2     """ LIFO Stack implementation using a Python list as underlying storage."""
3
4     def __init__(self):
5         """ Create an empty stack."""
6         self._data = []                # nonpublic list instance
7
8     def __len__(self):
9         """ Return the number of elements in the stack."""
10        return len(self._data)
11
12    def is_empty(self):
13        """ Return True if the stack is empty."""
14        return len(self._data) == 0
15
16    def push(self, e):
17        """ Add element e to the top of the stack."""
18        self._data.append(e)           # new item stored at end of list
19
```

Array-based Stack in Python

```
20 def top(self):
21     """Return (but do not remove) the element at the top of the stack.
22
23     Raise Empty exception if the stack is empty.
24     """
25     if self.is_empty():
26         raise Empty('Stack is empty')
27     return self._data[-1]          # the last item in the list
28
29 def pop(self):
30     """Remove and return the element from the top of the stack (i.e., LIFO).
31
32     Raise Empty exception if the stack is empty.
33     """
34     if self.is_empty():
35         raise Empty('Stack is empty')
36     return self._data.pop( )      # remove last item from list
```

9

Parentheses Matching

Each “(”, “{”, or “[” must be paired with a matching “)”, “}”, or “]”

correct: ()(()){([())}

incorrect:)((){([())}

correct: ((()))((){([())}

incorrect: ({ []})

incorrect: (

Parentheses Matching in Python

```
def is_matched(expr):
    lefty = '([{'
    # respective closing delims
    righty = ')]}'

    S = ArrayStack()

    for c in expr:
        if c in lefty:
            S.push(c) # push left delimiter on stack
        elif c in righty:
            if S.is_empty():
                return False # nothing to match with
            if righty.index(c) != lefty.index(S.pop()):
                return False

    return S.is_empty()

expr = '{[(5+2)*5+(34-5)/2]}'
print (is_matched(expr))
```

11

Questions

What values are returned during the following series of stack operations, if executed upon an initially empty stack?

push(5), push(3), pop(), push(2), push(8), pop(), pop(), push(9), push(1), pop(), push(7), push(6), pop(), pop(), push(4), pop(), pop().

Suppose an initially empty stack S has executed a total of 25 push operations, 12 top operations, and 10 pop operations, 3 of which raised Empty errors that were caught and ignored. What is the current size of S?

Infix, Prefix, and Postfix Notations

Infix Notation : $a*(b+c)$

Prefix Notation (Polish Notation) : $*a+bc$

Postfix Notation (Reverse Polish Notation) : $abc+*$

There is no need of parentheses for prefix and postfix notations

Dijkstra's Two-Stack Algorithm for Infix notation expressions

For each item of the infix expression:

1. A number: push it onto the value stack
2. An operator: push it onto the operator stack
3. Left parenthesis: ignore
4. Right parenthesis: Pop the operator from the operator stack. Pop the value stack twice, getting two operands. Apply the operator to those operands, in the correct order and Push the result onto the value stack again.

Caveat: every binary operation should be enclosed in parenthesis
eg. $(5 + ((12 - 2) * (24 * 45)))$

Generic Two-Stack Algorithm for Infix notation expressions

I. While there are still items to read, get the next item

For each item of the infix expression:

1. A number: push it onto the value stack
2. A left parenthesis: Push onto operator stack
3. A right parenthesis: do the following

While item $\neq$ left parenthesis:

- Pop the operator from the operator stack.
- Pop the value stack twice, getting two operands.
- Apply the operator to the operands, in the correct order.
- Push the result onto the value stack.

Pop out the left parenthesis

Generic Two-Stack Algorithm for Infix notation expressions

4. An operator **op**:

a) While (operator stack $\neq$ empty, and the top of the operator stack has the same or greater precedence than **op**):

- Pop the operator from the operator stack.
- Pop the value stack twice, getting two operands.
- Apply the operator to the operands, in the correct order.
- Push the result onto the value stack.

b) Push **op** onto the operator stack.

Generic Two-Stack Algorithm for Infix notation expressions

II. Repeat till operator stack is empty,

- Pop the operator from the operator stack.
- Pop the value stack twice, getting two operands.
- Apply the operator to the operands, in the correct order.
- Push the result onto the value stack.

At this juncture the operator stack should be empty, and the value stack should have only one value in it, which is the final result.

Evaluation of a postfix expression using a single stack

For each item in postfix expression

 If an operand is found

 push it onto the stack

 End-If

 If an operator op is found

 Pop the stack and assign stack element as B

 Pop the stack and assign stack element as A

 Evaluate $A \text{ op } B$ using the operator just found.

 Push the resulting value onto the stack

 End-If

End-For

Pop the stack (this is the computed result)

Questions

How to you modify the previous algorithm for Postfix expressions if the inputs are of prefix expressions?

Questions

Convert the following infix expressions to postfix expressions:

- (a) $(A * B) - C$
- (b) $A + (B/C) - D \% E$
- (c) $(X * Y) - (W \% Z) + V$
- (d) $(A - B) * (C + (D/E) * F) * G$
- (e) $A + (((B * C) / (D + E) * F) / G) \% (H ^ J)$

Questions

Convert these postfix expressions into infix

- (a) $A \ B \ - \ C \ *$
- (b) $A \ B \ * \ C \ +$
- (c) $A \ B \ + \ C \ D \ - \ E \ F \ - \ * \ ^$
- (d) $A \ B \ C \ * \ D \ + \ /$
- (e) $X \ Y \ Z \ ^ \ A \ B \ + \ \% \ *$

Questions

Convert these prefix expressions into infix

- (a) $+ \ * \ A \ B \ C$
- (b) $+ \ A \ ^ \ B \ C$
- (c) $+ \ * \ A \ - \ * \ ^ \ B \ C \ D \ \% \ + \ X \ Y \ - \ E \ F \ G$
- (d) $/ \ + \ ^ \ X \ Y \ Z \ * \ A \ \% \ * \ B \ C \ D$
- (e) $+ \ * \ X \ Y \ + \ \% \ W \ Z \ V$

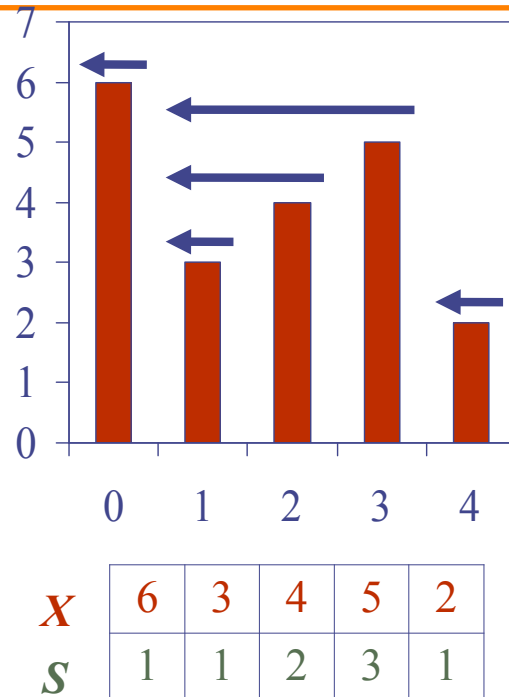
Computing Spans

Using a stack as an auxiliary data structure in an algorithm

Given an array X , the **span** $S[i]$ of $X[i]$ is the maximum number of consecutive elements $X[j]$ immediately preceding $X[i]$ and such that $X[j] \leq X[i]$

Spans have applications to financial markets, weather analysis, biological data analysis, and manufacturing quality control

E.g., stock at 52-week high



23

Quadratic Algorithm

Algorithm *spans1*(X, n)

Input array X of n integers

Output array S of spans of X

$S \leftarrow$ new array of n integers

for $i \leftarrow 0$ **to** $n - 1$ **do**

$s \leftarrow 1$

while $s \leq i$ && $X[i - s] \leq X[i]$

$s \leftarrow s + 1$

$S[i] \leftarrow s$

return S

#

n

n

n

$1 + 2 + \dots + (n - 1)$

$1 + 2 + \dots + (n - 1)$

n

1

◆ Algorithm *spans1* runs in $O(n^2)$ time

24

Computing Spans with a Stack

We keep in a stack the indices of the elements visible when “looking back”

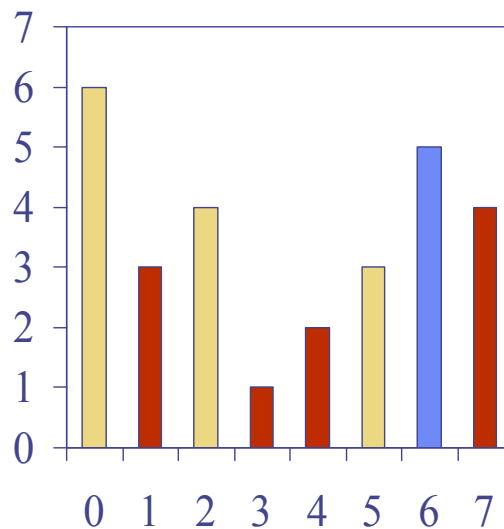
We scan the array from left to right

Let i be the current index

We pop indices from the stack until we find index j such that $X[i] < X[j]$

We set $S[i] \leftarrow i - j$

We push i onto the stack



25

Linear Algorithm

- ◆ Each index of the array
 - Is pushed into the stack exactly one
 - Is popped from the stack at most once
- ◆ The statements in the while-loop are executed at most n times
- ◆ Algorithm *spans2* runs in $O(n)$ time

```

Algorithm spans2( $X, n$ )           #
 $S \leftarrow$  new array of  $n$  integers   $n$ 
 $A \leftarrow$  new empty stack           1
for  $i \leftarrow 0$  to  $n - 1$  do       $n$ 
    while ( $\neg A.is\_empty()$  &&
            $X[A.top()] \leq X[i]$ ) do  $n$ 
         $A.pop()$                        $n$ 
    if  $A.is\_empty()$  then              $n$ 
         $S[i] \leftarrow i + 1$             $n$ 
    else
         $S[i] \leftarrow i - A.top()$      $n$ 
         $A.push(i)$                       $n$ 
return  $S$                             1
    
```

26

References

Data Structures and Algorithms in Python

Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser

Introduction to Algorithms

Leiserson, Stein, Rivest, Cormen

Algorithms, 4th Edition

Robert Sedgewick and Kevin Wayne

Few Images from the internet